

Partie 1

1. Pour les 4 problèmes ci-dessous, **éditer** (copier/coller interdit) et **tester** le programme en affichant les valeurs intermédiaires de toutes les variables (i compris). Exemple pour le problème 1, avec un print dans la boucle, vous devrez afficher la valeur de i, le temps en secondes et la vitesse instantanée.

Problème 1 : Soit à déterminer la vitesse instantanée en m/s d'un coureur à partir des temps intermédiaires.

bolt=[0.183,1.78,1.02,0.9,0.88,0.88,0.85,0.85,0.86,0.86,0.89,9.95]

Le premier temps est le temps de réaction du coureur. Le dernier temps est le temps cumulé et les valeurs intermédiaires sont les chronos intermédiaires mesurés tous les 10 mètres. Tous les temps sont en secondes.

Soit le script :

```
bolt=[0.183,1.78,1.02,0.9,0.88,0.88,0.85,0.85,0.86,0.86,0.89,9.95]
def vitesse_instantanee1(nom:[]):
    vit=[0]
    for i in range(1,len(nom)-1):
        vit.append(round(10/nom[i],2))
    return vit
print(vitesse_instantanee1(bolt))
```

Soit la variante qui affiche les vitesses en km/h.

```
bolt=[0.183,1.78,1.02,0.9,0.88,0.88,0.85,0.85,0.86,0.86,0.89,9.95]
def vitesse_instantanee2(nom:[]):
    vit=[0]
    for i in range(len(nom)-2):
        vit.append(round(3.6*10/nom[i+1],2))
    return vit
print(vitesse_instantanee2(bolt))
```

Problème 2 : Soit à déterminer la vitesse maximale en km/h d'un coureur à partir des vitesses intermédiaires.

Soit le script :

```
vit_bolt=vitesse_instantanee2(bolt)
vit_max=0
for vit in vit_bolt:
    if vit>vit_max:
        vit_max=vit
print("Vitesse maximale de Usain Bolt:",vit_max,"km/h")
```

Problème 3 : Soit à déterminer les accélérations instantanées en m/s^2 à partir des vitesses instantanées en m/s.

Soit le script :

```
vit_bolt=vitesse_instantanee1(bolt)
accel=[0]
for i in range(len(vit_bolt)-1):
    accel.append(round((vit_bolt[i+1]-vit_bolt[i])/bolt[i+1],2))
print(accel)
```

Soit le script qui généralise ce calcul d'accélération à partir de la liste des vitesses et celle de temps intermédiaires.

```
def accels(vit:[],chronos:[]):
    accel=[0]
    for i in range(len(vit)-1):
        accel.append(round((vit[i+1]-vit[i])/chronos[i+1],2))
    return accel
print(accels(vitesse_instantanee1(bolt),bolt))
```

Problème 4 : Soit à déterminer par intégration numérique l'énergie produite en Wh à partir des mesures de puissances instantanées relevées en Watt toutes les 5 minutes.

Soit le script :

```
""" Listes des valeurs de puissances relevées en Watt toutes les 5 minutes """
puissances=[10,10,20,55,80,60,96,75,50,68,60,45,24]
""" Calcul de l'énergie produite sur une heure par la méthode des trapèzes """
energie=0
for i in range(len(puissances)-1):
    energie=energie+5*(puissances[i+1]+puissances[i])/2
print ("Energie produite sur une heure en Wh: ",energie/60)
```

Partie 2

Soit l'extrait de sujet de SII TSI :



Figure 3 Réseau local à domicile

Contexte

Le vélo Pro-Form TDF (Tour de France), étudié ici, se distingue des vélos d'appartements classiques car il permet une immersion dans un environnement réaliste. Il est ainsi possible de reproduire les tracés réels d'une étape du Tour de France. Les écrans affichent généralement le rythme cardiaque et la distance parcourue sans oublier les calories brûlées à chaque entraînement ou encore la vitesse du cycliste. Grâce à ces statistiques, il est possible de cerner les faiblesses et les forces de l'utilisateur lors d'un parcours afin de permettre une progression continue.



Figure 1 Vélo Pro-Form TDF

Mise en situation

Ce vélo doit permettre de reproduire le plus fidèlement possible l'effort que doit fournir le cycliste en situation réelle. Vingt-quatre étapes du Tour de France ont été filmées et enregistrées avec les informations de dénivelé. Au rythme du défilement des images, le vélo va ajuster son inclinaison ainsi que la résistance au pédalage afin d'être conforme à la sensation et l'effort réel, comme le montre la figure 1.

Aussi il convient d'élaborer une loi de commande du dispositif de freinage qui va dépendre de la vitesse de rotation de la roue arrière et du dénivelé de la route souhaité.

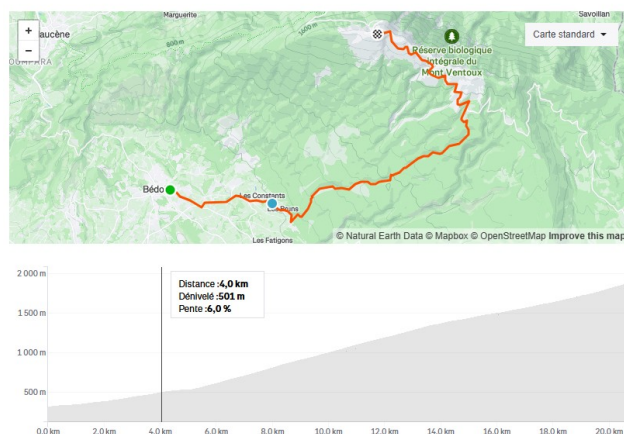
- Après avoir lu le sujet et la question Q6, **créer** et **tester** une fonction qui calcule la liste des pentes en % à partir des altitudes. Pour tester avec des valeurs réelles, **déclarer** une liste de valeurs d'altitude en vous reportant au site <https://www.strava.com/segments/38095101>. On se limitera aux valeurs tous les km.

Mont Ventoux (par Bédoin) ☆

Segment de vélo vérifié Bédoin, Vaucluse

Distance	Dénivelé positif	Pente moy.	Alt. la + basse	Alt. la + élevée	Dénivelé	Catégorie
20,74km	1569m	8,3 %	318m	1887m	1569m	HC

273 275 tentatives par 183 163 personnes



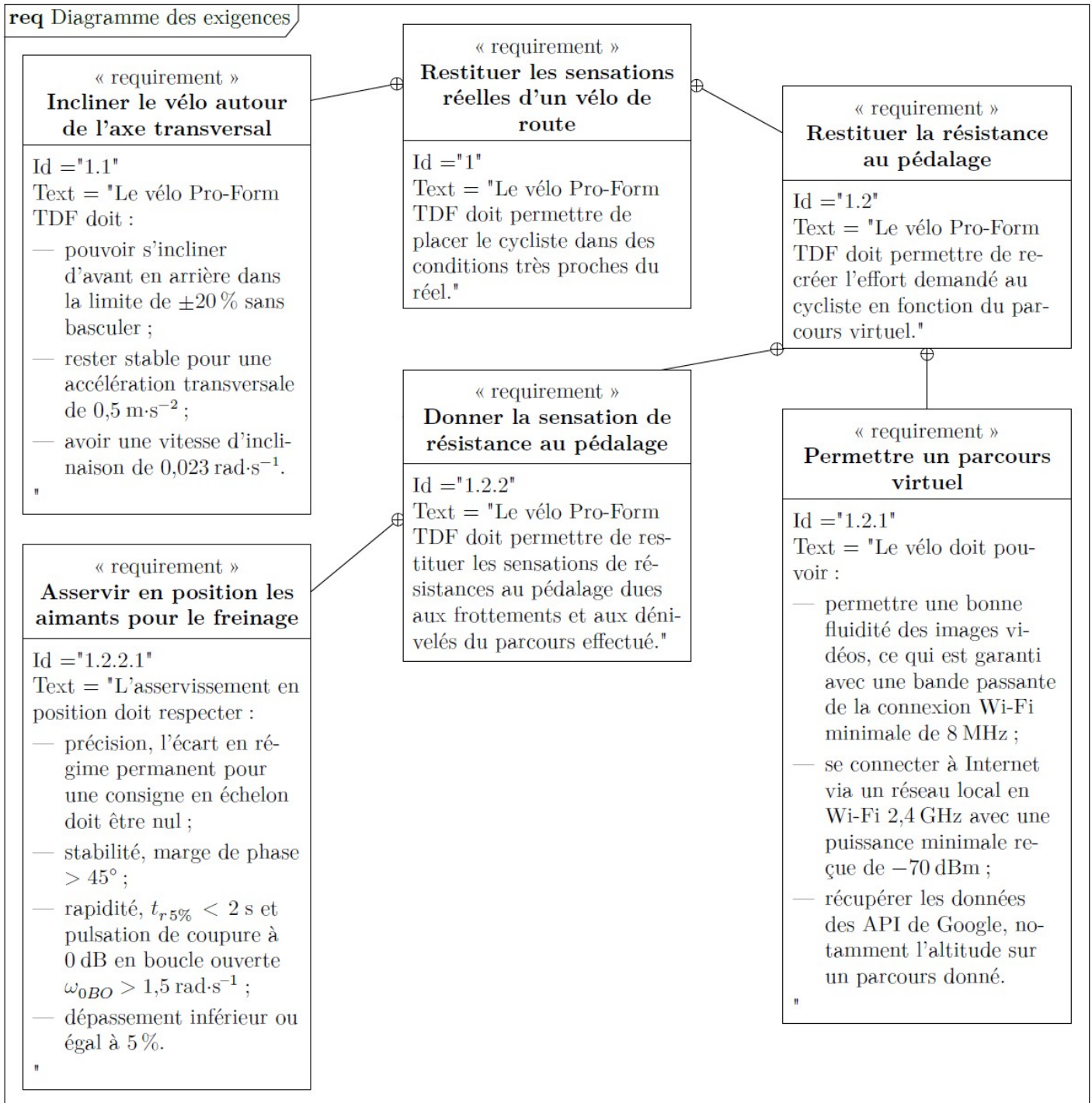


Figure 2 Exigences extraites du cahier des charges

*I.C – Récupération des données d'altitude à partir de Google Maps***Objectif**

Évaluer la possibilité de récupérer les pentes d'un parcours afin de commander le système d'inclinaison du vélo Pro-Form TDF.

Lorsque le vélo Pro-Form TDF est connecté à Internet, grâce aux services proposés par Google, il est possible de suivre un parcours en voyant les images de la route défiler comme si l'utilisateur était sur la route avec son vélo. Le vélo utilise pour cela les API (API est un acronyme anglais qui signifie « *Application Programming Interface* » ou Interface de Programmation d'Application) de Google et notamment l'API **Elevation** qui permet de récupérer l'altitude à partir de coordonnées géographiques.

Pour cela, une requête contenant les coordonnées géographiques des différents points du parcours est utilisée. Cette requête renvoie l'altitude de points régulièrement espacés (variable distance) le long du parcours. On s'intéresse, à titre d'exemple, à la montée du mont Ventoux en passant par Bédoin et le chalet Reynard. L'utilisateur du vélo Pro-Form TDF peut afficher le profil du parcours choisi comme illustré sur la figure 5.

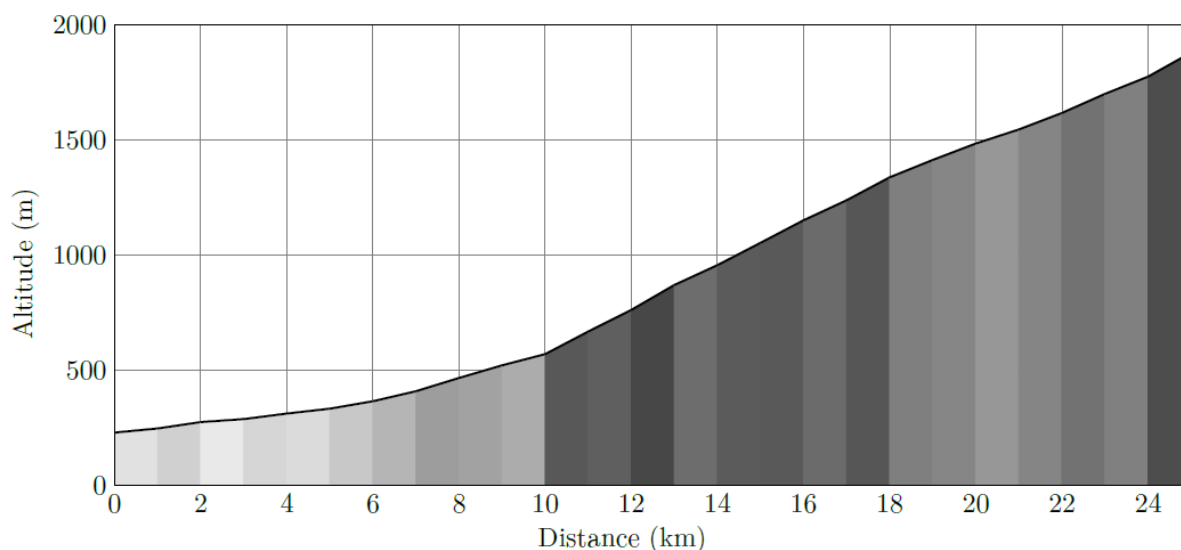


Figure 5 Profil de la montée du mont Ventoux

Afin de commander le dispositif d'élévation du vélo Pro-Form TDF, il est nécessaire de récupérer les différentes pentes, comme illustré sur la figure 6.

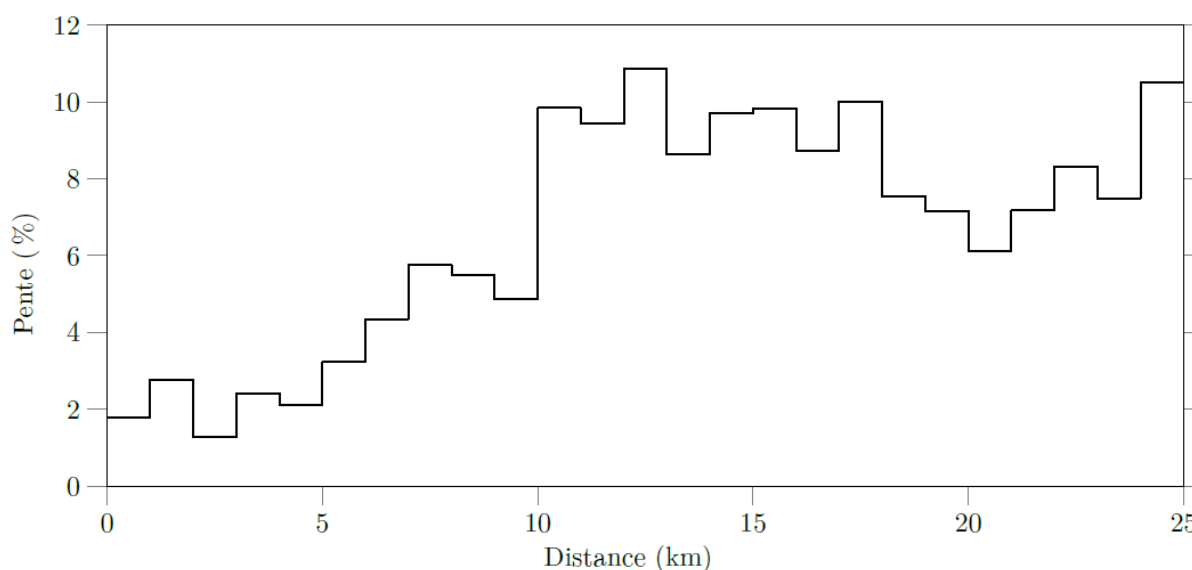


Figure 6 Pente moyenne par kilomètre lors de la montée du mont Ventoux

Q 6. En utilisant les API Google, on a obtenu l'altitude d'une suite de points $P_0, P_1, \dots, P_n$ régulièrement espacés le long du parcours de P_0 à P_n . Écrire en Python une fonction d'entête

```
def calculerPentes(distance:float, altitude:[float]) -> [float]:
```

qui prend en argument la distance le long du parcours entre deux points successifs quelconques P_i et P_{i+1} et une liste donnant l'altitude de chaque point P_i . Cette fonction renvoie une liste dont l'élément d'indice i donne la pente, en pourcent, de la portion du parcours située entre les points P_i et P_{i+1} .

Ainsi, si la connexion Internet est bonne, il est possible de récupérer des pentes tous les 100 m le long d'un parcours ce qui permet de suivre le plus fidèlement possible le trajet sur google maps. Maintenant que cette information de pente est disponible, il faut la traduire en résistance au pédalage ce qui est l'objet de la prochaine partie.